

Лабораторна робота №8.

Алгоритми хешування

Мета: створити простий сценарій Python для хешування даних.

Алгоритми хешування - це математичні функції, які перетворюють дані в хеш-значення фіксованої довжини, хеш-коди або хеші. Вихідне хеш-значення є буквально підсумком початкового значення. Найважливішим у цих хеш-значеннях є те, що неможливо отримати оригінальні вхідні дані лише з хеш-значень. Незважаючи на те, що шифрування є важливим для захисту даних (конфіденційності даних), іноді важливо мати можливість довести, що ніхто не змінював дані, які ви надсилаєте. Використовуючи значення хешування, ви зможете визначити, чи файл не змінювався після створення (цілісність даних). Насправді є багато прикладів використання хешування, включаючи цифрові підписи, шифрування відкритим ключем, автентифікацію повідомлень, захист паролем і багато інших криптографічних протоколів.

Завдання:

1. Імпортуйте **hashlib** - бібліотеку Python для хешування:

```
1  import hashlib
2
3  # encode it to bytes using UTF-8 encoding
4  message = "Some text to hash".encode()
5
```

2. Створіть хеш повідомлення використовуючи алгоритм MD5:

```
6  # hash with MD5 (not recommended)
7  print("MD5:", hashlib.md5(message).hexdigest())
8
```

3. Алгоритм MD5 застарів, тому його не слід використовувати, оскільки він не стійкий до злому. Використаємо SHA-2:

```
9  # hash with SHA-2 (SHA-256 & SHA-512)
10 print("SHA-256:", hashlib.sha256(message).hexdigest())
11
12 print("SHA-512:", hashlib.sha512(message).hexdigest())
```

SHA-2 - це сімейство з 4-х хеш-функцій: SHA-224, SHA-256, SHA-384 і SHA-512, ви також можете використовувати `hashlib.sha224()` і `hashlib.sha384()`. Проте в основному використовуються SHA-256 і SHA-512. Причина, чому він називається SHA-2 (Secure Hash Algorithm 2), полягає в тому, що SHA-2 є наступником SHA-1, який застарів і його легко зламати. SHA-2 генерує довші хеші, що призводить до вищого рівня безпеки, ніж SHA-1. Незважаючи на те, що SHA-2 все ще

використовується в наш час, багато хто вважає, що атаки на SHA-2 лише питання часу, дослідники стурбовані його довгостроковою безпекою через його схожість із SHA-1.

4. Використаємо SHA-3 - більш безпечний і повністю відмінний алгоритм хешування ніж SHA-2.

```
14 # hash with SHA-3
15 print("SHA-3-256:", hashlib.sha3_256(message).hexdigest())
16
17 print("SHA-3-512:", hashlib.sha3_512(message).hexdigest())
```

Існує небагато стимулів для оновлення до SHA-3, оскільки SHA-2 все ще безпечний, а оскільки швидкість також є проблемою (SHA-3 не швидший за SHA-2).

5. Якщо ми хочемо використовувати швидшу хеш-функцію, більш безпечну, ніж SHA-2, і принаймні таку ж безпечну, як SHA-3 то потрібно використовувати BLAKE2:

```
20 # hash with BLAKE2
21 # 256-bit BLAKE2 (or BLAKE2s)
22 print("BLAKE2c:", hashlib.blake2s(message).hexdigest())
23 # 512-bit BLAKE2 (or BLAKE2b)
24 print("BLAKE2b:", hashlib.blake2b(message).hexdigest())
```

Хешування алгоритмом BLAKE2 швидше, ніж SHA-1, SHA-2, SHA-3 і навіть MD5, і навіть безпечніше, ніж SHA-2. Він підходить для використання на сучасних процесорах, які підтримують паралельні обчислення в багатоядерних системах. BLAKE2 широко використовується та був інтегрований у основні криптографічні бібліотеки, такі як OpenSSL та Sodium тощо.

6. Створіть файл та за хешуйте його, просто прочитайте весь вміст файлу, а потім передайте байти файлу функціям хешування, які ми розглянули. Порівняйте результати роботи цих функцій.